



PRÁCTICA 11

MÓDULO BIOZON

SISTEMAS DE GESTIÓN EMPRESARIAL



19 DE ENERO DE 2026

LEYRE VELASCO CASADO

2ºDAM

Contenido

1. Introducción.....	2
2. Estado del arte	2
2.1 Definición de ERP	3
2.2 Uso de Odoo como ERP	3
2.3 Funcionamiento básico de Odoo	3
3. Descripción general del proyecto	4
3.1 Objetivos	4
3.2 Entorno de trabajo	4
4. Documentación técnica	5
4.1 Análisis del sistema	5
4.2 Diseño de la base de datos	9
4.3 Implementación	11
4.4 Pruebas de funcionamiento	14
4.5 Despliegue de la aplicación	18
5. Manuales	20
5.1 Manual de usuario	21
5.2 Manual de instalación.....	23
6. Conclusiones y posibles ampliaciones.....	25
7. Bibliografía	26

1. Introducción

Biozon es una tienda online especializada en la venta de productos naturales, ecológicos y respetuosos con la salud y el medio ambiente. La empresa basa su filosofía en promover un consumo responsable, ofreciendo productos libres de pesticidas, sustancias químicas nocivas y fabricados mediante procesos sostenibles. Su catálogo se organiza en tres grandes áreas: textil, alimentación y cosmética, lo que permite cubrir distintas necesidades del consumidor.

Uno de los valores más importantes de **Biozon** es su apoyo tanto a pequeños productores autónomos como a grandes empresas. En la plataforma conviven productos artesanales, como la miel de un apicultor de un pequeño pueblo de León, con productos de marcas consolidadas. Esta combinación permite a **Biozon** impulsar la economía local sin renunciar a una oferta variada y competitiva.

Desde el punto de vista de la gestión empresarial, **Biozon** necesita un sistema que permita controlar de forma centralizada toda la información relacionada con su actividad diaria. Gestionar productos, clientes, proveedores, pedidos y envíos de manera manual o con herramientas separadas puede provocar errores, duplicidad de datos y pérdida de tiempo. Por este motivo, se plantea el desarrollo de un módulo específico para Odoo que permita cubrir estas necesidades.

El objetivo de este proyecto es desarrollar un módulo completo y funcional para Odoo que permita gestionar de forma eficiente los elementos clave del negocio. Además, se incluye una API REST que permitirá en el futuro la integración con aplicaciones externas, ampliando las posibilidades de crecimiento de la plataforma.

La memoria se estructura en varios apartados. En primer lugar, se presenta el estado del arte, seguido de la descripción general del proyecto. A continuación, se desarrolla la documentación técnica, incluyendo análisis, diseño, implementación, pruebas y despliegue. Finalmente, se incluyen los manuales de usuario e instalación, así como las conclusiones del proyecto.

2. Estado del arte

El estado del arte es una parte del trabajo donde se explica cómo están actualmente las empresas en cuanto a digitalización y gestión. Sirve para ver qué

herramientas existen hoy en día, qué problemas intentan resolver y por qué cada vez más negocios necesitan sistemas que les ayuden a organizarse mejor. Gracias a esta visión general, se entiende por qué los ERP son tan importantes y por qué tiene sentido desarrollar un módulo como el de Biozon dentro de Odoo.

2.1 Definición de ERP

Un ERP (Enterprise Resource Planning) es un sistema de planificación de recursos empresariales que permite gestionar de forma integrada los principales procesos de una empresa. Estos sistemas centralizan la información en una única base de datos, facilitando la comunicación entre departamentos y mejorando la eficiencia en la toma de decisiones.

Antes del uso de los ERP, muchas empresas utilizaban programas independientes para cada área, lo que provocaba incoherencias y errores. Los ERP solucionan este problema al ofrecer una plataforma única y centralizada.

2.2 Uso de Odoo como ERP

Odoo es uno de los ERP de código abierto más utilizados actualmente. Se ha elegido para este proyecto porque permite desarrollar módulos personalizados que se adaptan a las necesidades específicas de una empresa. Además, utiliza tecnologías actuales como Python, XML y PostgreSQL, lo que facilita su aprendizaje y desarrollo.

Otra ventaja importante de Odoo es su arquitectura modular. Esto permite añadir nuevas funcionalidades sin modificar el núcleo del sistema, lo que lo convierte en una herramienta escalable y mantenible.

2.3 Funcionamiento básico de Odoo

Odoo se basa en una arquitectura Modelo–Vista–Controlador (MVC). Los modelos definen la estructura de los datos, las vistas determinan cómo se muestran al usuario y los controladores gestionan la lógica de la aplicación y las peticiones externas, como las APIs REST.

El uso del ORM de Odoo permite interactuar con la base de datos sin necesidad de escribir consultas SQL, lo que reduce errores y mejora la productividad.

3. Descripción general del proyecto

3.1 Objetivos

El objetivo principal de este proyecto es desarrollar un módulo completo y funcional para Odoo que permita a la tienda online **Biozon** gestionar de forma eficiente todos los elementos clave de su actividad: productos, categorías, clientes, proveedores, pedidos y envíos.

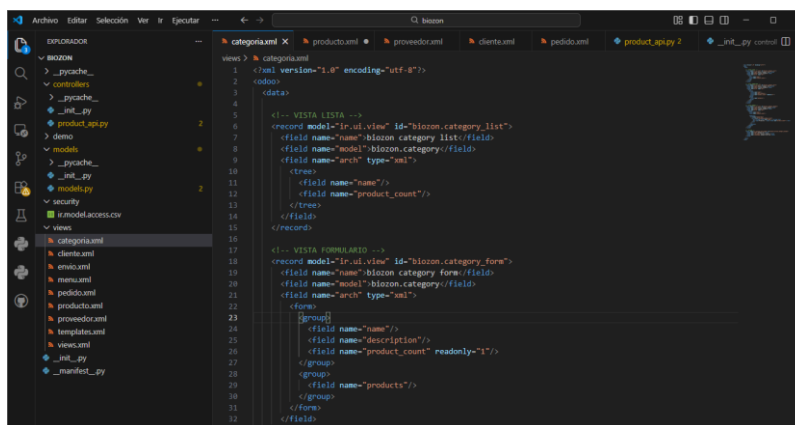
Para alcanzar este objetivo general, se han definido varios objetivos específicos, como diseñar una estructura de datos coherente y escalable, implementar vistas adaptadas al usuario, incorporar una API REST funcional, aplicar técnicas del ORM de Odoo y ofrecer una solución modular y mantenible.

3.2 Entorno de trabajo

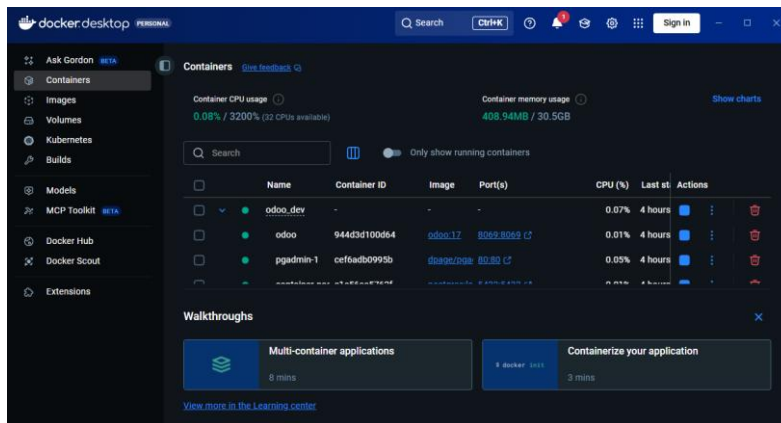
El desarrollo del módulo **Biozon** se ha realizado utilizando distintas herramientas y tecnologías. Odoo ha sido el ERP utilizado como base del proyecto, junto con Python para la lógica de negocio y XML para la definición de vistas. La base de datos utilizada ha sido PostgreSQL, que es la utilizada por defecto por Odoo.

Para el desarrollo del código se ha utilizado Visual Studio Code como editor, y Docker para ejecutar Odoo. Para las pruebas de la API REST se ha utilizado tanto el navegador.

Visual Studio Code



Docker



Odoo



4. Documentación técnica

La documentación técnica del módulo **Biozon** recoge y explica de forma detallada cómo se ha construido el proyecto desde dentro. En este apartado se describen los modelos creados con el ORM de Odoo, las vistas diseñadas en XML, los controladores utilizados para la API REST y las reglas de seguridad necesarias para que el sistema funcione correctamente. También se explica la estructura del módulo, cómo se organizan los archivos y qué relación existe entre las distintas partes del código. El objetivo de esta documentación es que cualquier persona que conozca Odoo pueda entender cómo está hecho el módulo, cómo funciona cada componente y cómo podría ampliarse o modificarse en el futuro.

4.1 Análisis del sistema

El desarrollo del módulo **Biozon** parte del análisis de las necesidades reales de la tienda online. **Biozon** gestiona un catálogo de productos naturales procedentes tanto de pequeños autónomos como de empresas consolidadas, por lo que necesita un sistema flexible y bien organizado.

Se identificaron necesidades como la gestión centralizada del catálogo, el control de proveedores, la gestión de clientes y pedidos, la gestión de envíos y la integración con aplicaciones externas mediante una API REST.

El desarrollo del módulo **Biozon** parte del análisis de las necesidades reales de la tienda online, que gestiona un catálogo de productos naturales y ecológicos procedentes tanto de pequeños autónomos como de empresas consolidadas. La plataforma requiere un sistema que permita organizar de forma eficiente la información relacionada con productos, clientes, proveedores, pedidos y envíos.

Necesidades de la empresa

Tras analizar cómo funciona **Biozon**, se pudieron identificar varias necesidades importantes que el sistema debía cubrir para mejorar la organización y evitar errores. La empresa trabaja con muchos productos diferentes, varios proveedores y clientes de toda España, por lo que era necesario contar con una herramienta que permitiera gestionar toda esta información de forma centralizada. Además, **Biozon** quería disponer de un sistema que facilitara el control del stock, la creación de pedidos y el seguimiento de los envíos, así como una forma sencilla de consultar los datos desde aplicaciones externas. A partir de este análisis se definieron las necesidades principales que debía cubrir el módulo desarrollado:

Gestión centralizada del catálogo

Biozon trabaja con tres grandes categorías de productos:

Textil, Alimentación, Cosmética

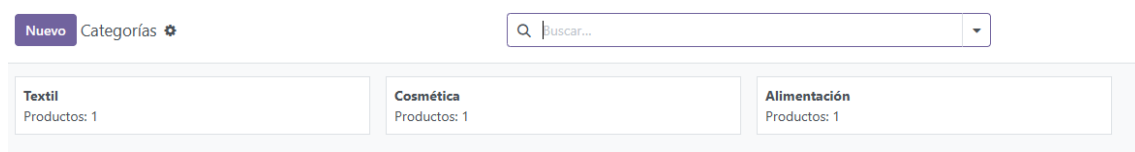
La empresa necesita un sistema que permita:

Registrar productos con sus características principales

Clasificarlos por categorías

Asociarlos a proveedores

Controlar el stock y detectar niveles bajos



Control de proveedores

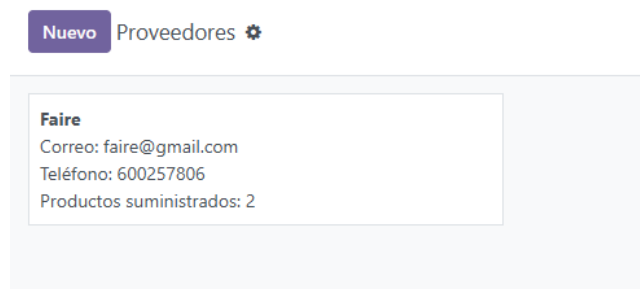
Biozon colabora con pequeños productores locales como el apicultor de León y con empresas más grandes.

Es necesario:

Registrar proveedores

Asociarlos a los productos que suministran

Conocer cuántos productos aporta cada uno



Gestión de clientes y pedidos

La tienda online recibe pedidos de clientes de toda España.

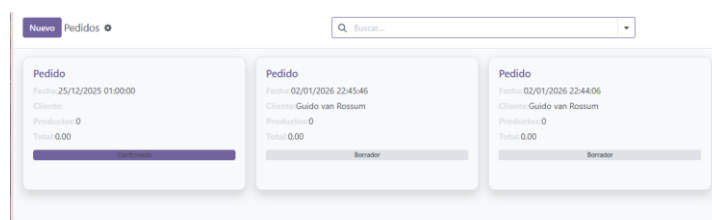
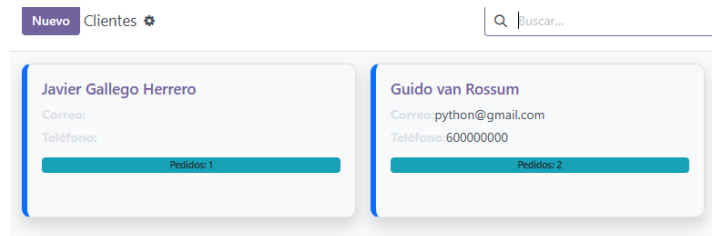
El sistema debe permitir:

Registrar clientes

Asociar pedidos a cada cliente

Calcular automáticamente el importe total

Controlar el estado del pedido (*borrado, confirmado, enviado, cancelado*)



Gestión de envíos

Cada pedido debe vincularse a un envío, que puede estar:

Pendiente

En tránsito

Entregado

Cancelado

Además, es importante conocer qué cliente corresponde a cada envío.

Pedido	Cliente	Estado	Fecha envío
biozon.order,1		En tránsito	
biozon.order,3	Guido van Rossum	Entregado	

Integración con aplicaciones externas

Biozon quiere crecer y permitir que otras aplicaciones consulten su catálogo.

Por ello, se requiere:

Una API REST sencilla

Que devuelva información en formato **JSON**

Accesible desde navegadores o herramientas como **Postman**

```
[{"name": "Camisa", "price": 100.0}, {"name": "Crema Hidratante Neutrogena", "price": 35.0}, {"name": "Pipas de calabaza", "price": 16.0}]
```

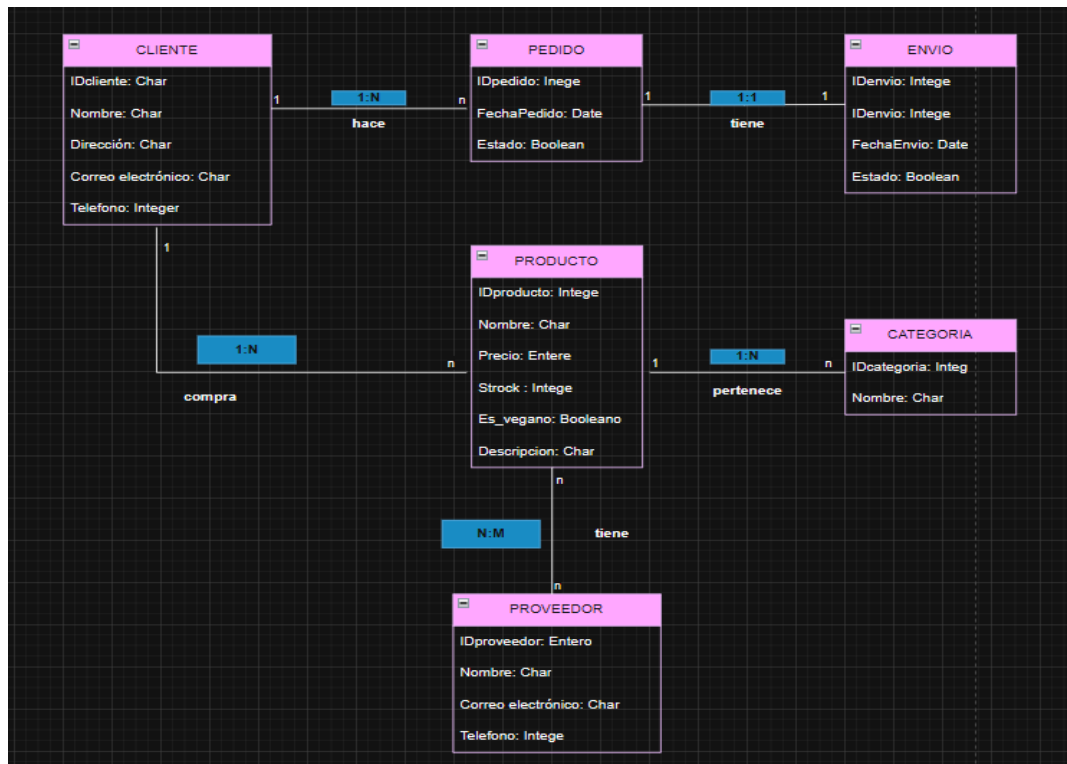
4.2 Diseño de la base de datos

El diseño de la base de datos del módulo **Biozon** se ha realizado utilizando el ORM de Odoo, lo que permite definir los modelos directamente en Python y que Odoo se encargue de la creación y gestión de las tablas en la base de datos PostgreSQL.

Para el desarrollo del módulo se han creado varios modelos que representan las principales entidades del negocio. En concreto, se han definido modelos para **categorías, productos, proveedores, clientes, pedidos y envíos**, elementos necesarios para el funcionamiento de la tienda online.

Las relaciones entre los modelos: un cliente puede realizar varios pedidos, cada pedido se asocia a un único envío y los productos pertenecen a una categoría. Además, un producto puede estar relacionado con uno o varios proveedores, lo que permite representar correctamente la colaboración con distintos proveedores.

Este diseño facilita la organización de los datos y evita duplicidades, además de permitir que el sistema sea fácil de mantener y ampliar en el futuro. Al utilizar el ORM de Odoo, se garantiza también la integridad de los datos y una correcta comunicación entre los distintos modelos del módulo.



Las relaciones entre los modelos permiten reflejar la realidad de la tienda online. Por ejemplo, una categoría puede tener varios productos, un producto puede tener

varios proveedores y un cliente puede realizar varios pedidos. Este diseño facilita la integridad de los datos y permite realizar consultas eficientes.

Relaciones:

RELACIÓN	TIPO
Cliente - Pedido	One2many
Pedido - Envío	One2one
Producto - Categoría	Many2one
Producto - Proveedor	Many2many

Relaciones en Producto:

```
category = fields.Many2one("biozon.category", string="Categoría")

# Relación N:M con proveedores
suppliers = fields.Many2many(
    comodel_name="biozon.supplier",
    relation="products_suppliers",
    column1="product_id",
    column2="supplier_id",
    string="Proveedores"
)
```

Relaciones en Categoría:

```
# Relación 1:N con productos
products = fields.One2many(
    comodel_name="biozon.product",
    inverse_name="category",
    string="Productos"
)
```

*En estas relaciones (capturas) observamos como se realizan las relaciones Many2one, Many2many, One2many en **visual studio code**

4.3 Implementación

La implementación del módulo se ha realizado siguiendo la estructura estándar de Odoo. siguiendo la estructura básica de los módulos de este. El módulo se ha organizado en diferentes carpetas para separar correctamente los modelos, las vistas y los controladores, lo que facilita la comprensión del código y su mantenimiento.

En la parte de **modelos**, se han creado las clases necesarias utilizando Python y el ORM de Odoo. En estos modelos se han definido los campos principales de cada entidad, como el nombre del producto, el precio, el stock y la categoría, así como las relaciones entre las distintas tablas mediante campos *Many2one*, *One2many* y *Many2many*. Gracias al uso del ORM, no ha sido necesario escribir consultas SQL manuales.

Las **vistas** del módulo se han definido mediante archivos XML. Se han creado vistas de tipo lista (*tree*), formulario (*form*) y kanban para facilitar la gestión de los datos desde la interfaz de Odoo. Estas vistas permiten al usuario crear, modificar y consultar los registros de forma sencilla y visual.

Además, se ha implementado un **controlador REST** que permite acceder a la información de los productos desde fuera de Odoo. Este controlador expone un endpoint que devuelve los datos en formato JSON,

(implementación de PRODUCTO)

```
# PRODUCTO
class Product(models.Model):
    _name = 'biozon.product'
    _description = 'Producto'

    # Campos principales
    name = fields.Char(string="Nombre")
    description = fields.Text(string="Descripción")
    price = fields.Float(string="Precio")
    stock = fields.Integer(string="Stock")
    is_vegan = fields.Boolean(string="¿Es vegano?")
    category = fields.Many2one("biozon.category", string="Categoría")

    # Relación N:M con proveedores
    suppliers = fields.Many2many(
        comodel_name="biozon.supplier",
        relation="products_suppliers",
        column1="product_id",
        column2="supplier_id",
        string="Proveedores"
    )

    # Campo computado: stock bajo
    low_stock = fields.Boolean(
        string="¿Stock bajo?",
        compute="_compute_low_stock",
        store=True
    )

    image = fields.Image(string="Imagen")
```

```
# MÉTODO ORM EXPLÍCITO
# Usa search() y write()

def update_low_stock_products(self):
    """
    Busca todos los productos cuyo stock es menor a 10
    y actualiza su campo low_stock a True.
    Este método demuestra el uso explícito del ORM.
    """
    low_stock_products = self.env['biozon.product'].search([('stock', '<', 10)])
    low_stock_products.write({'low_stock': True})
    return True
```

Memoria Biozon

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <data>

    <!-- VISTA TREE (LISTA)-->

    <record id="biozon.product_list" model="ir.ui.view">
      <field name="name">biozon product list</field>
      <field name="model">biozon.product</field>
      <field name="arch" type="xml">
        <tree>
          <!-- Campos principales del producto -->
          <field name="name"/>
          <field name="internal_code"/>
          <field name="price"/>
          <field name="stock"/>
          <field name="category"/>
        </tree>
      </field>
    </record>
  </data>
</odoo>
```

```
<!-- VISTA FORM (FORMULARIO) -->

<record id="biozon.product_form" model="ir.ui.view">
  <field name="name">biozon product form</field>
  <field name="model">biozon.product</field>
  <field name="arch" type="xml">
    <form>
      <!-- Grupo principal con los datos del producto -->
      <group>
        <field name="name"/>
        <field name="description"/>
        <field name="price"/>
        <field name="stock"/>
        <field name="is_vegan"/>
        <field name="category"/>
        <field name="image" widget="image"/>
      </group>

      <!-- Grupo secundario con los proveedores -->
      <group>
        <field name="suppliers"/>
      </group>
    </form>
  </field>
</record>
```

```
<!-- HERENCIA DE VISTA -->

<record id="biozon.product_form_inherit" model="ir.ui.view">
  <field name="name">biozon.product.form.inherit</field>
  <field name="model">biozon.product</field>

  <!-- Heredamos la vista original -->
  <field name="inherit_id" ref="biozon.product_form"/>

  <field name="arch" type="xml">
    <!-- Insertamos el campo después del nombre -->
    <xpath expr="//field[@name='name']" position="after">
      <field name="internal_code"/>
    </xpath>
  </field>
</record>
```

```
<!-- ACCIÓN DEL MENÚ -->

<record id="biozon.action_product_window" model="ir.actions.act_window">
  <field name="name">Productos</field>
  <field name="res_model">biozon.product</field>
  <field name="view_mode">kanban,tree,form</field>
</record>

</data>
</odoo>
```

```
<!-- VISTA KANBAN (CON BOOTSTRAP) -->

<record id="biozon.product_kanban" model="ir.ui.view">
  <field name="name">biozon product kanban</field>
  <field name="model">biozon.product</field>

  <field name="arch" type="xml">
    <kanban>
      <!-- Campos cargados en el kanban -->
      <field name="name"/>
      <field name="internal_code"/>
      <field name="price"/>
      <field name="stock"/>
      <field name="is_vegan"/>
      <field name="category"/>
      <field name="low_stock"/>
      <field name="image"/>

      <templates>
        <!-- Plantilla de cada tarjeta del kanban -->
        <t t-name="kanban-box">
          <!-- Contenedor principal con Bootstrap -->
          <div class="oe_kanban_global_click card shadow-sm p-3 mb-2"
              style="border-radius: 10px; background-color: #f8f9fa;">
            <!-- Imagen del producto -->
            <div class="text-center mb-2">
              
            </div>

            <!-- Nombre del producto -->
            <h4 class="text-primary fw-bold">
              <field name="name"/>
            </h4>
          </div>
        </t>
      </templates>
    </kanban>
  </field>
</record>
```

Uso del ORM de Odoo

Para gestionar los datos del módulo Biozon se ha utilizado el ORM (Object Relational Mapping) de Odoo, que permite interactuar con la base de datos mediante objetos Python en lugar de escribir consultas SQL manuales. Gracias a este sistema, las operaciones de búsqueda, creación y actualización de registros se realizan de forma sencilla, segura y totalmente integrada con la estructura del framework.

A continuación, se muestran los métodos del ORM utilizados en el módulo:

```
@api.depends('stock')
def _compute_low_stock(self):
    for product in self:
        product.low_stock = product.stock is not False and product.stock < 10
```

```
# MÉTODO ORM EXPLÍCITO
# Usa search() y write()

def update_low_stock_products(self):
    """
    Busca todos los productos cuyo stock es menor a 10
    y actualiza su campo low_stock a True.
    Este método demuestra el uso explícito del ORM.
    """
    low_stock_products = self.env['biozon.product'].search([('stock', '<', 10)])
    low_stock_products.write({'low_stock': True})
    return True
```

Código del controlador product_api.py.

A continuación, se muestra el controlador REST implementado para exponer los datos de los productos en formato JSON. Este controlador permite acceder a la información desde aplicaciones externas mediante una ruta pública.

```
controllers > product_api.py > ...
1  from odoo import http      No se ha podido resolver la importación "odoo".
2  from odoo.http import request, Response  No se ha podido resolver la importación "odoo.http".
3  import json
4
5  class ProductAPI(http.Controller):
6
7      @http.route('/api/products', auth='public', type='http', methods=['GET'], csrf=False)
8      def get_products(self):
9          products = request.env['biozon.product'].search([])
10         data = [{'name': p.name, 'price': p.price} for p in products]
11         return Response(json.dumps(data), content_type='application/json')
12
13
```

```
3  access_biozon_product,biozon.product,model_biozon_product,base.group_user,1,1,1,1
```

*En esta fase se han implementado los modelos, las vistas, los controladores y las reglas de seguridad necesarias para el correcto funcionamiento del sistema.

4.4 Pruebas de funcionamiento

Para comprobar el correcto funcionamiento del módulo **Biozon** se han realizado distintas pruebas sobre las principales funcionalidades implementadas. Estas pruebas se han llevado a cabo tanto desde la interfaz de Odoo como mediante el uso de la API REST desarrollada.

En primer lugar, se realizaron pruebas sobre la API REST de productos, accediendo al endpoint correspondiente desde el navegador y desde herramientas externas. En estas pruebas se comprobó que la información de los productos se devolvía correctamente en formato JSON, incluyendo los datos principales como el nombre, el precio y el stock.

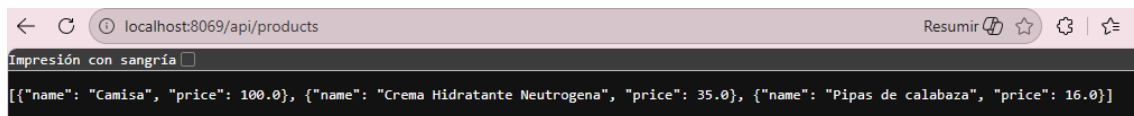
También se realizaron pruebas desde la interfaz de Odoo, creando nuevos productos y comprobando que se guardaban correctamente en la base de datos. Durante estas pruebas se verificó el funcionamiento del control de stock, activándose el indicador de stock bajo en aquellos productos que no alcanzaban el mínimo establecido.

Otra de las pruebas realizadas fue la creación de pedidos, comprobando que el sistema calculaba automáticamente el importe total en función de los productos

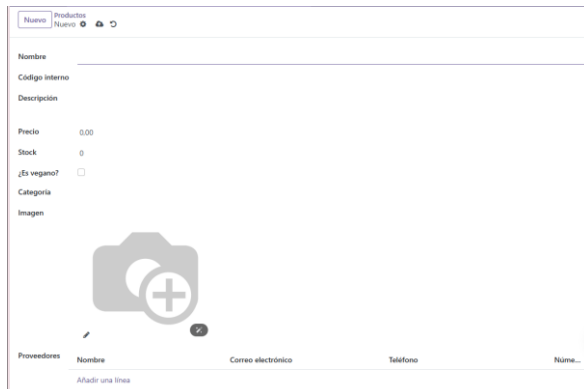
añadidos. A continuación, se verificó los pedidos con los envíos, comprobando que cada pedido quedaba vinculado a su correspondiente envío y que el estado del mismo se actualizaba correctamente.

Tras realizar todas estas pruebas, se pudo comprobar que el módulo funciona de forma correcta y cumple con los requisitos definidos durante el análisis del sistema.

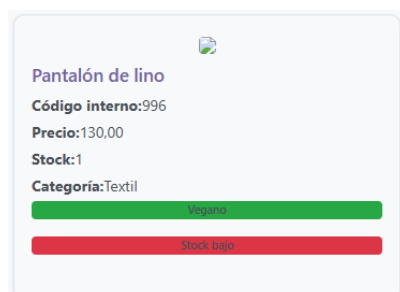
Prueba del endpoint /api/products devolviendo datos JSON.



Creación de un producto desde la interfaz de Odoo.



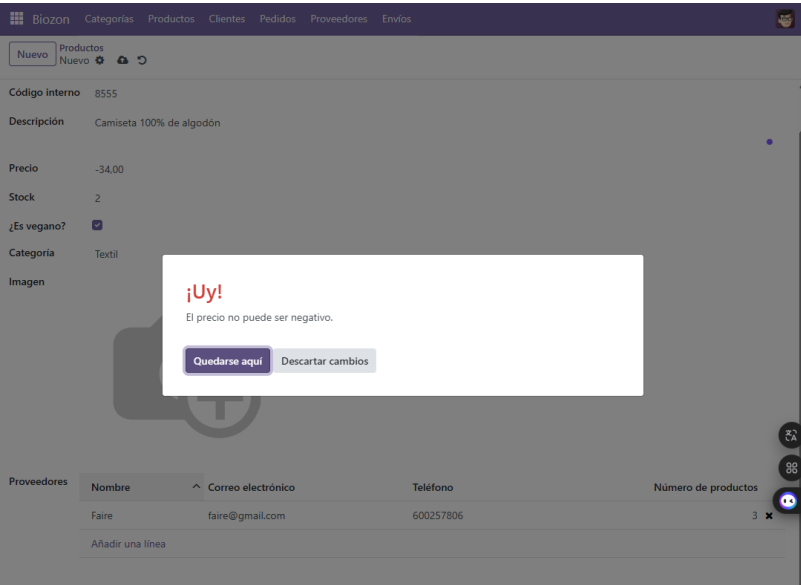
Activación del campo low_stock en productos con stock bajo.



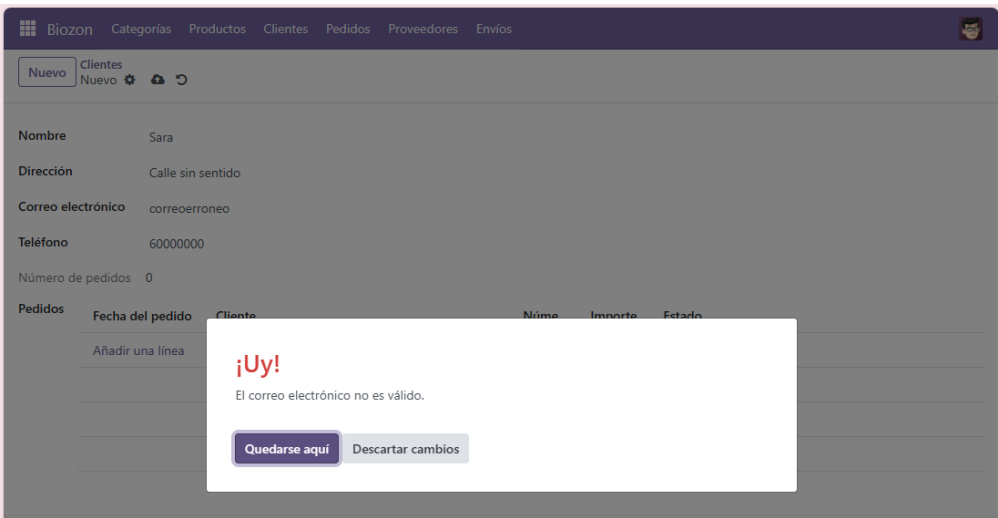
Stock $\geq$ 10 $\rightarrow$ low_stock = False

Stock < 10 $\rightarrow$ low_stock = True

Producto con precio negativo:



Validación de correo electrónico:



Validación de número de teléfono:

The screenshot shows the 'Nuevo' (New) client form in the Biozon application. The form fields are: Nombre (Name), Dirección (Address), Correo electrónico (Email), and Teléfono (Phone). The 'Teléfono' field contains the value 'a9i'. A modal dialog box is displayed in the center of the screen with the text '¡Uy!' and 'El teléfono solo puede contener números.' (The phone number can only contain numbers). Below the message are two buttons: 'Quedarse aquí' (Stay here) and 'Descartar cambios' (Discard changes).

Nombre: Guido van Rossum
Dirección: Calle de la Programación 1
Correo electrónico: python@gmail.com
Teléfono: a9i
Número de pedidos: 0

¡Uy!
El teléfono solo puede contener números.

Quedarse aquí Descartar cambios

Validación de nombre:

The screenshot shows the 'Nuevo' (New) client form in the Biozon application. The form fields are: Nombre (Name), Dirección (Address), Correo electrónico (Email), and Teléfono (Phone). The 'Nombre' field is empty. A modal dialog box is displayed in the center of the screen with the text '¡Uy!' and 'El nombre no puede estar vacío.' (The name cannot be empty). Below the message are two buttons: 'Quedarse aquí' (Stay here) and 'Descartar cambios' (Discard changes).

Nombre:
Dirección: Calle de la Programación 1
Correo electrónico: python@gmail.com
Teléfono:
Número de pedidos: 0

¡Uy!
El nombre no puede estar vacío.

Quedarse aquí Descartar cambios

Pedido con cálculo automático del total.

Nuevo Pedido

Fecha del pedido: 15/01/2026 14:13:50

Cliente: Javier Gallego Herrero

Estado: Borrador

Número de productos: 2

Importe total: 135,00€

Productos	Nombre	Código interno	Precio	Stock	Categoría
	Camisa	789	100,00	25	Textil
	Crema Hidratante Neutrogena		35,00	25	Cosmética
Añadir una línea					

Envío: biozon.shipment,1

Envío asociado a un pedido.

Nuevos Envíos

Buscar...

Pedido: biozon.order,1 Cliente: Estado: En tránsito Fecha envío:	Pedido: biozon.order,3 Cliente: Guido van Rossum Estado: Entregado Fecha envío:
---	--

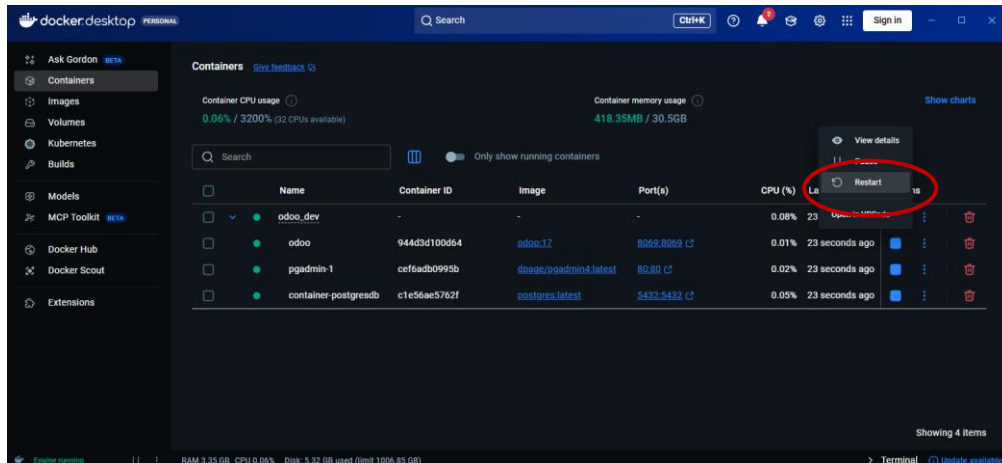
4.5 Despliegue de la aplicación

El despliegue del módulo **Biozon** se ha realizado en un entorno local utilizando Docker, lo que permite ejecutar Odoo y la base de datos PostgreSQL de forma aislada y controlada.

Para ello, se ha utilizado un archivo docker-compose, en el que se definen los servicios necesarios para el funcionamiento de la aplicación, incluyendo el contenedor de Odoo y el de la base de datos. Una vez configurado el entorno, se ha iniciado el despliegue mediante comandos ejecutados desde la terminal.

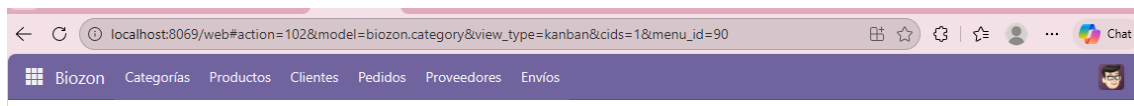
Tras levantar los contenedores, se accedió a Odoo desde el navegador web (localhost:8069) y se comprobó que el sistema funcionaba correctamente.

A continuación, se instaló el módulo Biozon desde el panel de aplicaciones de Odoo, verificando que no se producían errores durante la instalación y que todas las funcionalidades estaban disponibles.



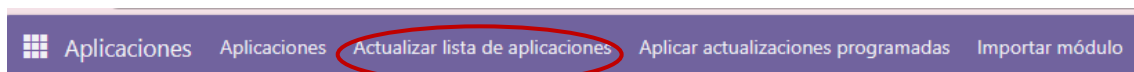
Contenedores de Odoo y PostgreSQL en ejecución en Docker. Siempre para levantar la aplicación y que todo funcione correcto, debemos restart a todos los contenedores de Odoo como se indica en la captura

A continuación:



Acceso a Odoo desde el navegador tras iniciar el contenedor.

Como conseguimos ver **Biozon**:



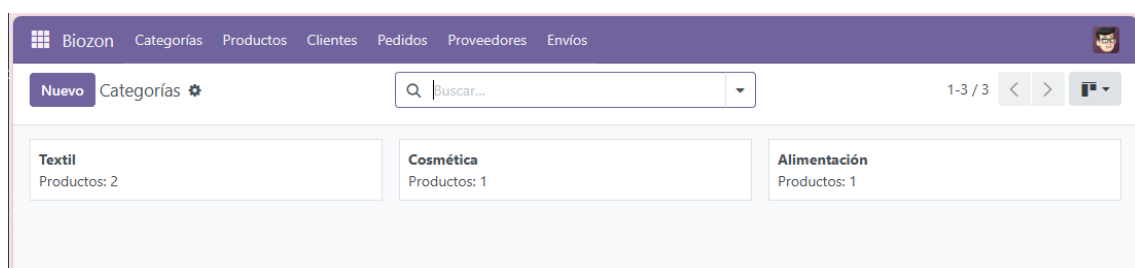
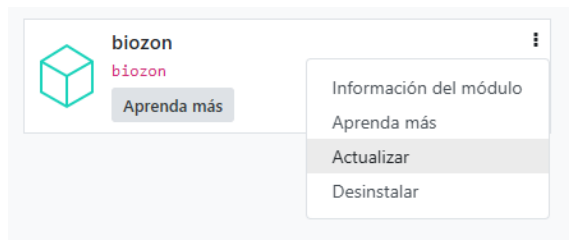
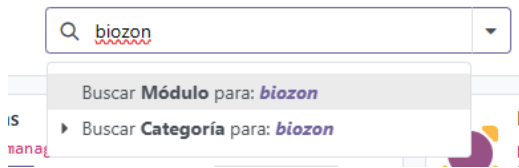
Se deben de actualizar la lista de aplicaciones

Actualización del módulo

Haga clic en Actualizar para iniciar el proceso...

Actualizar Cancelar

Una vez actualizamos buscamos el contenedor en la lupa o buscador:



Módulo **Biozon** instalado y funcionando en Odoo

5. Manuales

Los manuales incluidos en este proyecto tienen como objetivo explicar de forma clara cómo utilizar el módulo **Biozon** y cómo instalarlo correctamente en un entorno local. El manual de usuario describe las diferentes vistas del módulo, cómo crear y gestionar productos, categorías, clientes, proveedores, pedidos y envíos, y muestra capturas de pantalla para que el uso sea más intuitivo. Por otro lado, el manual de instalación detalla los pasos necesarios para poner en marcha el módulo en Odoo utilizando Docker, desde la preparación del entorno hasta la instalación final del módulo. Ambos manuales están pensados para que cualquier persona, incluso sin experiencia previa en Odoo, pueda seguirlos sin dificultad.

5.1 Manual de usuario

El módulo **Biozon** permite gestionar productos, categorías, clientes, pedidos, proveedores y envíos desde la interfaz de Odoo.

Vista tree: Lista de productos con columnas clave, Muestra los productos en forma de tabla con columnas clave como nombre, categoría, precio y stock.

```
<!-- VISTA TREE (LISTA)-->

<record id="biozon.product_list" model="ir.ui.view">
  <field name="name">biozon product list</field>
  <field name="model">biozon.product</field>
  <field name="arch" type="xml">
    <tree>
      <!-- Campos principales del producto -->
      <field name="name"/>
      <field name="internal_code"/>
      <field name="price"/>
      <field name="stock"/>
      <field name="category"/>
    </tree>
  </field>
</record>
```

Vista form: Formulario para crear o editar productos

```
<!-- VISTA FORM (FORMULARIO) -->

<record id="biozon.product_form" model="ir.ui.view">
  <field name="name">biozon product form</field>
  <field name="model">biozon.product</field>
  <field name="arch" type="xml">
    <form>
      <!-- Grupo principal con los datos del producto -->
      <group>
        <field name="name"/>
        <field name="description"/>
        <field name="price"/>
        <field name="stock"/>
        <field name="is_vegan"/>
        <field name="category"/>
        <field name="image" widget="image"/>
      </group>

      <!-- Grupo secundario con los proveedores -->
      <group>
        <field name="suppliers"/>
      </group>
    </form>
  </field>
</record>
```

Vista Kanban: Tarjetas visuales con imagen, nombre, precio, stock facilitando la identificación rápida.

```
<!-- VISTA KANBAN (CON BOOTSTRAP) -->

<record id="biozon.product_kanban" model="ir.ui.view">
  <field name="name">biozon product kanban</field>
  <field name="model">biozon.product</field>

  <field name="arch" type="xml">
    <kanban>

      <!-- Campos cargados en el kanban -->
      <field name="name"/>
      <field name="internal_code"/>
      <field name="price"/>
      <field name="stock"/>
      <field name="is_vegan"/>
      <field name="category"/>
      <field name="low_stock"/>
      <field name="image"/>

      <templates>

        <!-- Plantilla de cada tarjeta del kanban -->
        <t t-name="kanban-box">

          <!-- Contenedor principal con Bootstrap -->
          <div class="oe_kanban_global_click card shadow-sm p-3 mb-2"
              style="border-radius: 10px; background-color: #f8f9fa;">

            <!-- Imagen del producto -->
            <div class="text-center mb-2">
              
            </div>

            <!-- Nombre del producto -->
            <h4 class="text-primary fw-bold">
              <field name="name"/>
            </h4>

          </div>

        </t>

      </templates>

    </kanban>
  </field>
</record>
```

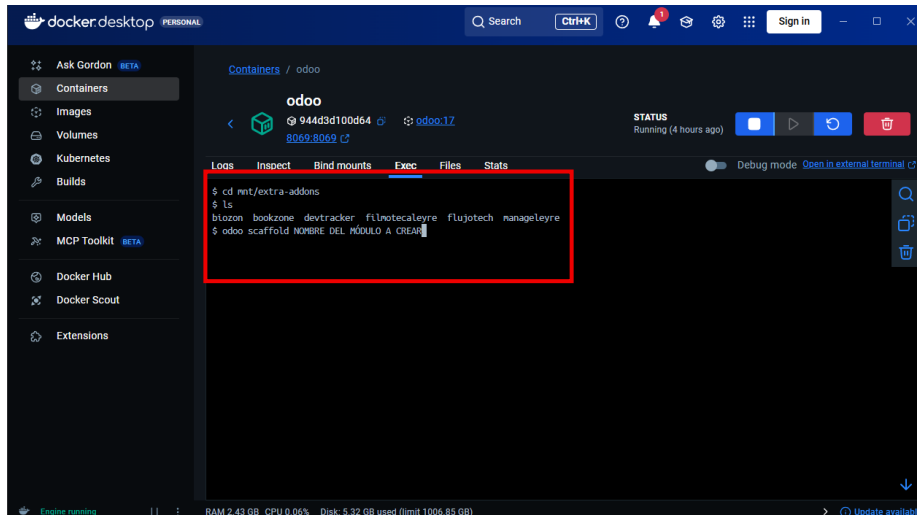
El usuario puede gestionar fácilmente los registros mediante estas vista

5.2 Manual de instalación

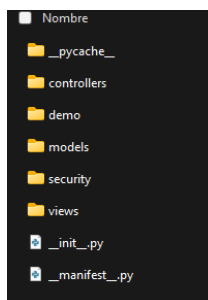
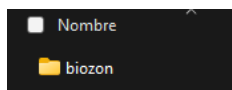
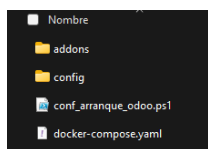
El primer paso consiste en instalar Docker Desktop desde su página oficial. Una vez instalado, Docker permite ejecutar Odoo y PostgreSQL sin necesidad de configuraciones manuales.

Pasos:

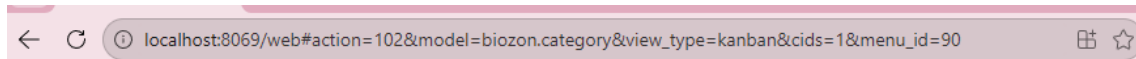
En la terminal de Odoo:



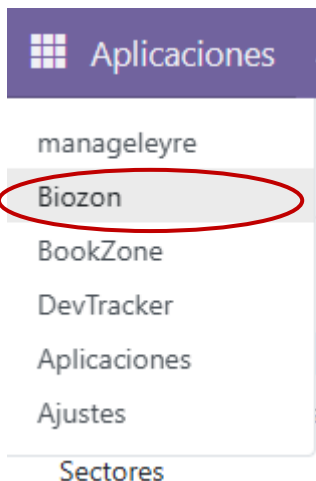
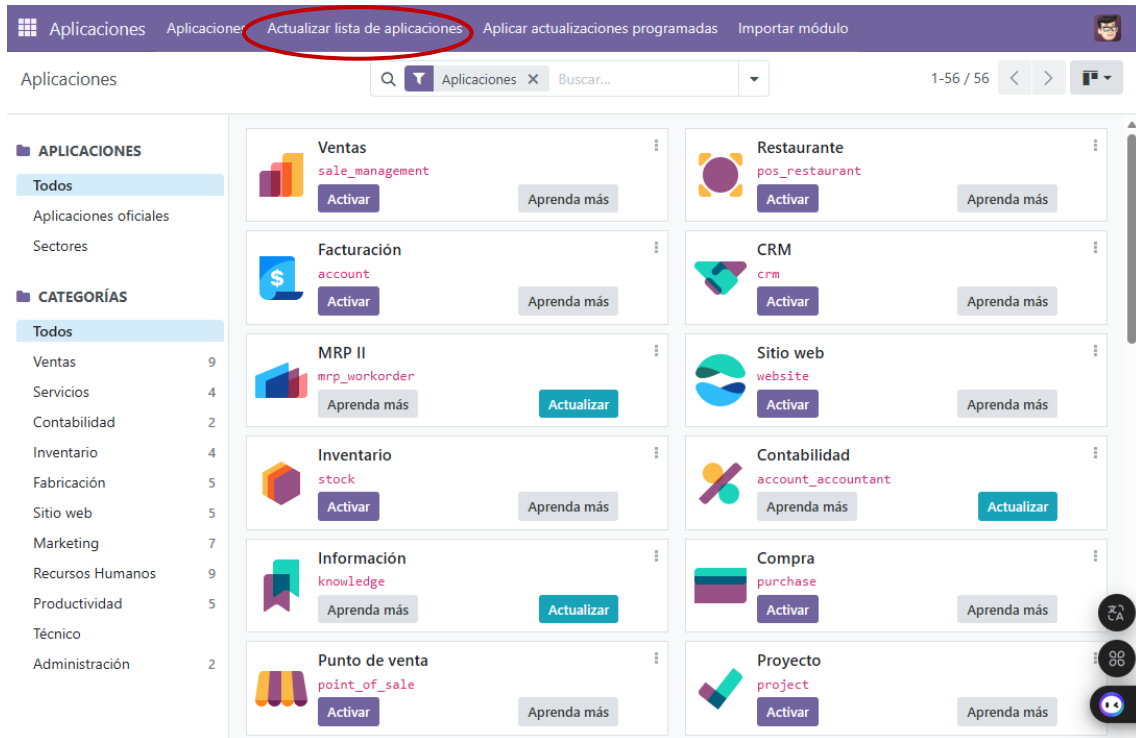
Una vez hecho esto se crea automáticamente la carpeta de **Biozon**



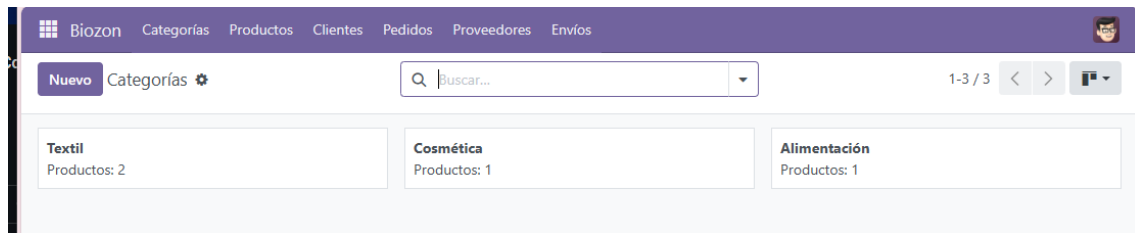
Para verificar que la instalación ha sido correcta, en el navegador ponemos localhost:8090 siendo 8090 el puerto correspondiente a Odoo



Nos aparecerá la interfaz de Odoo, en la cual haremos una actualización de las aplicaciones:



Nos saldrá el módulo de **Biozon** listo para activar, una vez activado estamos dentro y podemos ver su interfaz:



6. Conclusiones y posibles ampliaciones

El desarrollo del módulo **Biozon** me ha permitido poner en práctica todo lo aprendido sobre Odoo durante el curso. Gracias a este proyecto he podido trabajar con los elementos principales del ERP, como los modelos, las vistas, el ORM, los controladores y las reglas de seguridad. Esto me ha ayudado a entender mejor cómo funciona Odoo por dentro y cómo se estructura un módulo real.

Aunque al inicio surgieron dificultades, especialmente en la organización del módulo, en la creación de la API REST y en la personalización visual de la vista kanban donde ajustar colores y estilos resultó más complejo de lo esperado, el proceso me ha servido para afianzar conocimientos y aprender a resolver problemas por mi cuenta. También he podido comprobar la importancia de seguir una estructura ordenada y de probar cada parte del sistema para asegurar que todo funciona correctamente.

El resultado final es un módulo funcional que permite gestionar productos, categorías, clientes, proveedores, pedidos y envíos, además de ofrecer una API sencilla para consultar información desde aplicaciones externas.

De cara al futuro, el módulo se podría mejorar añadiendo nuevas funcionalidades, como:

- Un sistema de facturación integrado.
- Métodos de pago para completar el ciclo de venta.
- Paneles de análisis con gráficos y estadísticas.
- Más endpoints en la API para crear o modificar datos.
- Automatizaciones, como avisos de stock bajo o pedidos automáticos a proveedores.

7. Bibliografía

- Apuntes del módulo Sistemas de Gestión Empresarial
- Herramienta **Draw.io** para la realización de diagrama
- <https://youtu.be/QXJCCo9bQC0>
- Para corrección de textos y verificación IA Copilot